

# ASSIGNMENT 10 – SQL

**Use the Sakila Database in MySQL workbench and frame the SQL queries for below scenarios.**

- Q1. Write a query to display the first name, last name, and rental date of all customers who rented a film. Use the customer and rental tables.

```
select first_name, last_name, rental_date
from rental
join customer
using(customer_id);
```

- Q2. Write a query to display the first name, last name, and address of all customers whose district is either Alberta or Texas. Use the customer and address tables.

```
select first_name, last_name, address
from customer
join address
using(address_id)
where district in ('Alberta','Texas');
```

- Q3. List all films along with their category names, including films that may not belong to any category. Then sort it by category names. Use the film, film\_category and category tables.

```
select title, name
from film
left join film_category
using(film_id)
left join category
using(category_id)
order by name;
```

Q4. Display the number of rentals per store. Use the rental, inventory, and store tables.

```
select store_id, count(distinct rental_id) as count_rental
from rental
join inventory
using(inventory_id)
join store
using(store_id)
group by store_id;
```

Q5. List the names of categories that have more than 50 films. Use table category and film\_category.

```
select distinct name
from (select name, count(distinct film_id) as film_count
from category
join film_category
using(category_id)
group by category_id
having film_count > 50
order by film_count desc) b
order by name;
```

Q6. Write a query to display the film titles that have been rented by the customer named 'Mary Smith'. Use a subquery to get the customer ID.

```
select distinct title
from film
join inventory
using(film_id)
join rental
using(inventory_id)
where customer_id in (
select distinct customer_id
from customer where first_name = 'Mary' and
last_name = 'Smith');
```

Q7. Write a query to display the total rental income generated by each film. Use film, inventory, rental and payment tables.

```
select title,sum(amount) as income
from film
join inventory
using(film_id)
join rental
using(inventory_id)
join payment
using(rental_id)
group by 1
order by income desc;
```

Q8. Write a query to display the total number of films each actor has acted in. Use actor, film\_Actor tables.

```
select concat(first_name, " ", last_name) as actor_name,
       count(distinct film_id) as film_count
from actor
join film_actor
using(actor_id)
group by 1
order by film_count desc;
```

Q9. Write a query to list all rentals with a case statement that displays 'New Customer' if the customer\_id is greater than 300, and 'Regular Customer' otherwise. Use table rental.

```
select rental_id, customer_id,case when customer_id > 300 then 'New Customer'
else 'Regular Customer' end as tag
from rental
group by rental_id, customer_id, tag
order by customer_id desc;
```

Q10. Write a query to find all films with titles that start with the letter 'A'. Display the film title and the release year. Use film table and inventory table.

```
select title, release_year
from film
join inventory
using(film_id)
where title like 'A%'
group by title, release_year;
```

Q11. Create a new DB and perform below operations using SQL commands.

- (a) Create a table with 2 columns – Student\_ID, Marks
- (b) Insert 2 records in this table.
- (c) Rename the Marks column with Score.
- (d) Then delete the table.
- (e) Then delete the Database.

```
CREATE DATABASE StudentDB;
USE StudentDB;
CREATE TABLE Students (
    Student_ID INT PRIMARY KEY,
    Marks INT
);
INSERT INTO Students (Student_ID, Marks) VALUES (1, 85);
INSERT INTO Students (Student_ID, Marks) VALUES (2, 90);
ALTER TABLE Students RENAME COLUMN Marks TO Score;
DROP TABLE Students;
DROP DATABASE StudentDB;
```